

Patrones de Arquitectura

Maestro-Esclavo: Construcción de Sistemas Distribuidos Escalables

Patrones de Arquitectura Maestro-Esclavo: Construcción de Sistemas Distribuidos Escalables

La replicación de bases de datos es uno de los desafíos más antiguos de los sistemas distribuidos, y la arquitectura maestro-esclavo (ahora a menudo llamada primary-replica) sigue siendo una de las soluciones más prácticas para escalar cargas de trabajo con muchas lecturas. Pero no es una solución universal.

El Problema: Punto Único de Estrangulamiento de Lectura

Imagina que has construido un microservicio que alimenta el motor de recomendaciones de tu plataforma. Cada acción del usuario—una búsqueda, una vista, un clic—desencadena lecturas de base de datos. Tu base de datos puede manejar escrituras bien, pero el tráfico de lectura sigue aumentando. Escalas horizontalmente agregando servidores de aplicaciones, pero todos golpean la misma base de datos. La base de datos se convierte en tu cuello de botella, y agregar más servidores no ayuda.

Entra el patrón maestro-esclavo: una base de datos primaria acepta escrituras, y una o más réplicas manejan lecturas. Tu aplicación escribe en la primaria y lee de las réplicas, distribuyendo la carga de lectura entre múltiples máquinas.

Cómo Funciona

```
// Pseudo-código mostrando la responsabilidad dividida
public class DatabaseRouter {
    private Connection primary; // Punto final de escritura
    private List replicas; // Puntos finales de lectura

    public void insertOrder(Order order) { // Todas las escrituras van a primaria
        primary.executeUpdate("INSERT INTO orders ...");
    }

    public Order getOrder(String id) { // Las lecturas pueden ir a cualquier réplica
        Connection readDb = loadBalanceReplicas();
        return readDb.query("SELECT * FROM orders WHERE id = ?");
    }
}
```

La primaria replica cambios a sus réplicas usando un registro binario (MySQL), registro de escritura adelantada (PostgreSQL), o un mecanismo equivalente. Las réplicas aplican estos cambios de forma asincrónica, por lo que son eventualmente consistentes—no inmediatamente consistentes. Ese es el compromiso.

Cuándo Usarlo (Y Cuándo No)

Usa maestro-esclavo cuando: - Tu carga de trabajo es pesada en lecturas (80%+ lecturas) - Tienes capacidad de sobra en las réplicas durante horas valle - La consistencia eventual es aceptable - Estás usando una base de datos que soporta replicación de forma nativa (MySQL, PostgreSQL, MongoDB)

No lo uses cuando: - Tus lecturas y escrituras están balanceadas (¿por qué replicar?) - Necesitas garantías de consistencia fuerte (aplicaciones de estricta consistencia como procesamiento de pagos) - La latencia de replicación importa—milisegundos cuentan en escenarios de trading de alta frecuencia - Intentas resolver escalabilidad de escritura (la replicación solo ayuda con lecturas)

Advertencias del Mundo Real

La latencia de replicación es real. Un usuario crea una orden en la primaria, presiona "Actualizar", y lee de una réplica que aún no se ha puesto al día. Ve "Orden no encontrada". Necesitas lógica de reintentos, estrategias de invalidación de caché, o enrutamiento de escrituras de vuelta a la primaria por un breve período.

El failover es desordenado. Si tu primaria muere, promover una réplica es una decisión de gestión, no automática. ¿Cuál réplica debería convertirse en primaria? ¿Está completamente actualizada, o perderás datos? ¿Te perdiste una escritura urgente? Los frameworks populares como Spring Data abstraen parte de esto, pero el problema permanece.

El monitoreo es innegociable. Métricas de latencia de replicación, comprobaciones de salud primaria/réplica, y uso de disco del registro de escritura adelantada todos necesitan paneles. Perder una latencia de replicación cada vez mayor hasta que esté horas atrasada es un modo de fallo común.

Ejemplo con Spring Boot

```
@Configuration public class DataSourceConfig {
    @Bean public DataSource primaryDataSource() {
        return DataSourceBuilder.create()
            .url("jdbc:mysql://primary.db.example.com/mydb")
            .username("appuser").password("pass").build();
    }
    @Bean public DataSource replicaDataSource() {
        return DataSourceBuilder.create()
            .url("jdbc:mysql://replica.db.example.com/mydb")
            .username("appuser").password("pass").build();
    }
    @Bean public JdbcTemplate writeTemplate(DataSource primary) {
        return new JdbcTemplate(primary);
    }
    @Bean public JdbcTemplate readTemplate(DataSource replica) {
        return new JdbcTemplate(replica);
    }
}
```

Luego inyecta la plantilla correcta basándote en la operación:

```
@Service public class OrderService { @Autowired private JdbcTemplate writeTemplate;
@Autowired private JdbcTemplate readTemplate; public void saveOrder(Order o) {
writeTemplate.update("INSERT INTO orders ..."); } public Order findOrder(String id) { return
readTemplate.query("SELECT * FROM orders WHERE id = ?"); } }
```

El Cambio Moderno: Réplicas de Lectura Sobre Maestro-Esclavo

Las bases de datos en la nube actuales (RDS, Cloud SQL, Azure Database) a menudo usan la terminología "réplicas de lectura" en lugar de "esclavo"—en parte por sensibilidad, en parte porque la división de responsabilidad es más explícita en un servicio gestionado. No gestionas el protocolo de replicación tú mismo; lo hace el proveedor de la nube. Solo aprovisionas réplicas y apuntas tu aplicación a ellas.

Conclusión Clave

La arquitectura maestro-esclavo es un patrón elegante para escalar lecturas, pero es una solución táctica, no un panacea arquitectónico. Emparéjala con monitoreo sólido, lógica de reintentos, y garantías de consistencia claras en tu código de aplicación. Cuando la latencia de replicación o el failover se convierte en un punto de dolor frecuente, podrías estar superando este patrón—considera sharding, capas de caché, u ofertas de base de datos como servicio que manejen estas preocupaciones por ti.

¿Qué desafíos de replicación has experimentado en producción? Comparte tu experiencia abajo.